

COMS3011A Project 1

Brendan Griffiths

1 Tutor Management System

The school of Computer Science and Applied Mathematics is trying to come up with a better way to manage their tutors. They need a platform that allows an organiser to assign the various available tutors to different courses based on a few criteria: The tutor's marks in the course, their availability with their own timetable, and hours per week. This platform also needs to provide a way for students to volunteer for any overflow work, sign and submit their timesheets, excuse themselves and more admin capabilities.

1.1 Features

1.1.1 Basic

The platform should hold the school's courses and its tutors, with each tutor's marks in a course, the hours they can work in a week, and the times they are already busy.

Assigning tutors to courses is the core of the platform. An organiser should work from a single board rather than a sequence of forms, and the platform should answer for them as they go: whether a tutor's marks qualify them for a course, whether it clashes with their timetable, and how many of their weekly hours are left. A choice that breaks one of these should be visible as it is made, not after it is saved.

Tutors should log the hours they work, submit them as a timesheet, and have an organiser sign them off. A tutor should be able to excuse themselves from a session they cannot make, and a student should be able to volunteer for work nobody has taken.

Each role needs its own way in: an allocation board for the organiser, a dashboard telling a tutor what they owe and when, and a plain view for a student wanting to volunteer. Nobody should have to learn how the platform is arranged to find the one thing they came to do.

1.1.2 Intermediate

The platform should start doing the organiser's legwork. It should rank the tutors who could take a course, fill an obvious set of assignments in bulk, and show where a course is still short, while letting an organiser override any of it with a reason recorded.

Availability should be easier to give than to describe, so a tutor should import their timetable, set recurring exceptions, and have it read against the term calendar.

Timesheets should become a workflow, passing through approval, carrying corrections and disputes, prompting whoever is holding one up, and leaving in a form payroll can use. Overflow work should be

posted where volunteers can claim it and an organiser can approve the claim. Anything that changes an assignment, or falls due, should reach the person it affects.

An organiser should be able to see hours per course, spend against budget, and how the work is spread across tutors.

Above all the platform should stay quick to use as it fills up: guided onboarding for a first-time tutor, a search that reaches any course, person, or timesheet, keyboard navigation for an organiser living in the board all day, and bulk actions wherever the repetitive ones are.

1.1.3 Advanced

The platform should allocate rather than only validate. Given the courses, the tutors, and their constraints, it should propose an allocation across the whole school, balance objectives that pull against one another, and leave the assignments an organiser has locked alone. Its reasoning has to be legible: for any assignment it should say why this tutor, and for any tutor it passed over, why not. This is the hardest part of the platform, and it is only worth having if an organiser trusts it enough to accept what it proposes.

Allocation is rarely done alone, so several organisers should be able to work the same board at once, see where each other are, and have conflicting edits resolved rather than overwritten.

Before committing to anything, an organiser should be able to draft allocations side by side, compare them, and publish the one they settle on.

Tutors should be able to trade sessions between themselves, with the platform checking a swap against the same constraints and routing it for approval.

Finally, every change should be recorded, so the platform can say who moved what, and an allocation can be restored to an earlier state.

AI Declaration: The preceding document was generated with: Claude-Code[Claude Opus 4.8]